Tutorials Exercises W3Schools app iOS & Android NEW Sign In
. CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

React Forms - Multiple Input Fields

‹ Previous Next ›

Handling Multiple Inputs

SHARE

When you have multiple controlled input fields in a form, you can manage their state either by:

1. Using a separate useState call for each input.

2. Using a single useState call with an object to hold all form field values.

We will use the second approach, as it is more common for forms.

Make sure each input field has a unique name attribute.

Also, when initializing the state, use an object instead of a string. If the input fields have no initial value, use an empty object.

Example:

Get your own React.js Server

Use the `useState` Hook to manage the input:

Tutorials Exercises
W3Schools app NEW iOS & Android
Sign In
. CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

```
function MyForm() {
  const [inputs, setInputs] = useState({});

  const handleChange = (e) => {
    const name = e.target.name;
    const value = e.target.value;
    setInputs(values => ({...values, [name]: value}))
  }

  return (
    <form>
      <label>First name:
      <input
        type="text"
        name="firstname"
        value={inputs.firstname}
        onChange={handleChange}
      />
      </label>
      <label>Last name:
        <input
          type="text"
          name="lastname"
          value={inputs.lastname}
          onChange={handleChange}
        />
      </label>
      <p>Current values: {inputs.firstname} {inputs.lastname}</p>
    </form>
  )
}

createRoot(document.getElementById('root')).render(
  <MyForm />
);
```

Tutorials Exercises W3Schools app iOS & Android NEW Sign In
CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

Example Explained:

The first thing to notice is that when using a single useState call, we use an object to hold any initial values. In this case, with no initial values, we use an empty object:

```
const [inputs, setInputs] = useState({});
```

Next, the handleChange function is updated to handle multiple input fields.

In the function, we access the input fields in the event handler using the `e.target.name` and `e.target.value` syntax.

To update the state, use square brackets [bracket notation] around the property name.

```
function handleChange(e) {
  const name = e.target.name;
  const value = e.target.value;
  setInputs(values => ({...values, [name]: value}))
}
```

When refering to input values, we add the name of the state object, `inputs`, as well as the name of the input field:

```
<input
  type="text"
  name="firstname"
  value={inputs.firstname}
  onChange={handleChange}
/>
<input
  type="text"
  name="lastname"
  value={inputs.lastname}
  onChange={handleChange}
```

Tutorials Exercises W3Schools app iOS & Android NEW Sign In
CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

REMOVE ADS

Initial Values

To add initial values to the input fields in the example above, add the proper keys and values to the `useState` object:

Example:

Use initial values for firstname and lastname:

```
function MyForm() {
  const [inputs, setInputs] = useState({
    firstname: 'John',
    lastname: 'Doe'
  });

  ...
```

Tutorials Exercises W3Schools app iOS & Android NEW Sign In

. CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

‹ Previous

Sign in to track progress

Next ›

Build skills.
Get hired.

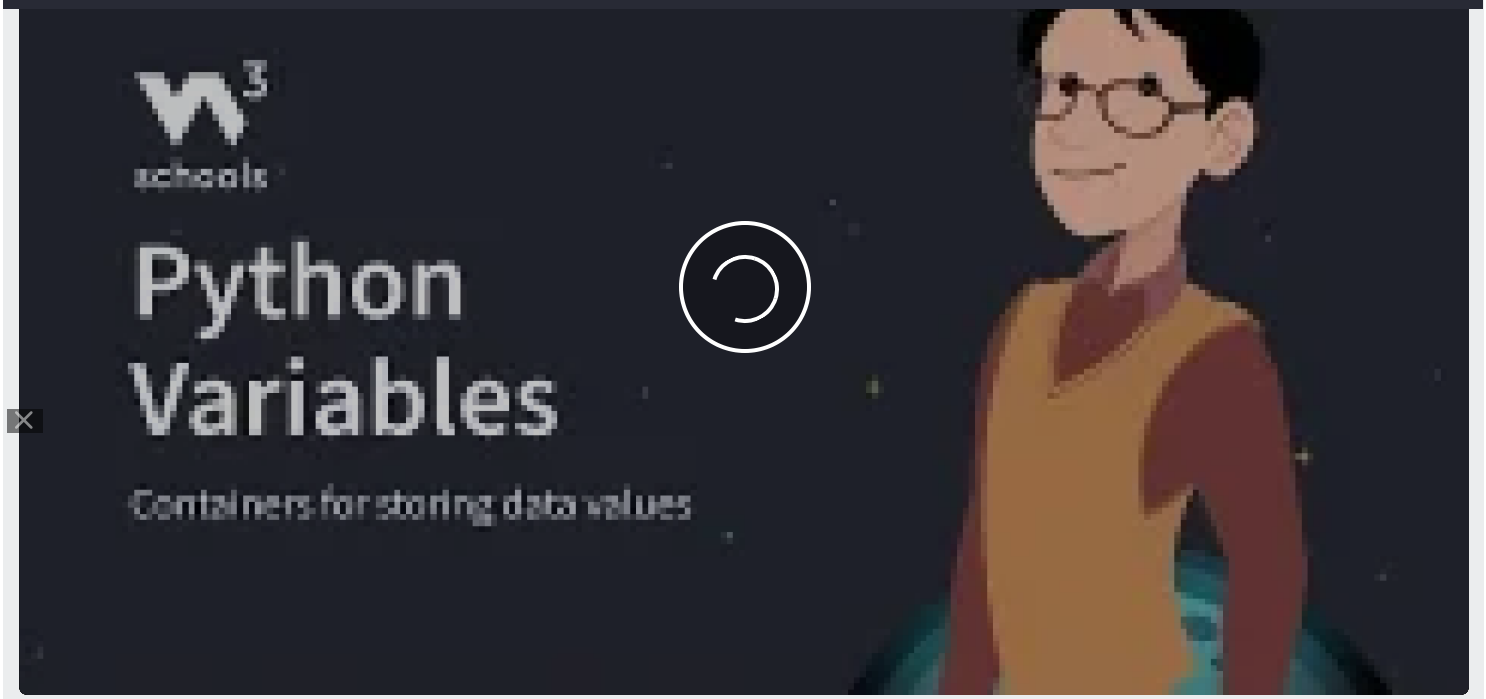
$95 per paid course
one-time payment

Exam and certificate included.

Get started

REMOVE ADS

Tutorials Exercises
W3Schools app NEW iOS & Android
Sign In
. CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS



Tutorials Exercises W3Schools app iOS & Android NEW Sign In

CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

FREE ACCOUNT

Track your progress

XP earned 0

Day streak 0

Spaces 0

League —

Sign in to track progress

NEW

Learn in Adventure Mode

Coding fundamentals as bite-sized lessons and challenges.

Download on the App Store GET IT ON Google Play

Learn more about Adventure Mode >

PLUS SPACES

GET CERTIFIED FOR TEACHERS

PRACTICE CONTACT US

Top Tutorials

HTML Tutorial
CSS Tutorial
JavaScript Tutorial
How To Tutorial
SQL Tutorial
Python Tutorial
W3.CSS Tutorial
Bootstrap Tutorial
PHP Tutorial
Java Tutorial

Tutorials Exercises W3Schools app iOS & Android NEW Sign In

CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

CSS Reference
JavaScript Reference
SQL Reference
Python Reference
W3.CSS Reference
Bootstrap Reference
PHP Reference
HTML Colors
Java Reference
AngularJS Reference
jQuery Reference

Top Examples

HTML Examples
CSS Examples
JavaScript Examples
How To Examples
SQL Examples
Python Examples
W3.CSS Examples
Bootstrap Examples
PHP Examples
Java Examples
XML Examples
jQuery Examples

Get Certified

HTML Certificate
CSS Certificate
JavaScript Certificate
Front End Certificate
SQL Certificate
Python Certificate
PHP Certificate
jQuery Certificate
Java Certificate
C++ Certificate
C# Certificate
XML Certificate

FORUM ABOUT CAMPUS

W3Schools is optimized for learning and training. Examples might be simplified to improve reading and learning.
Tutorials, references, and examples are constantly reviewed to avoid errors, but we cannot warrant full correctness
of all content. While using W3Schools, you agree to have read and accepted our terms of use, cookies and privacy policy.

Copyright 1999-2026 by Refsnes Data. All Rights Reserved. W3Schools is Powered by W3.CSS.